# iProc Big List

Last edit: 2026.03.05 by J. Segawa

*How to read these notes:*

<u>iPROC STEP FLAG</u>

- **function in steps.py**
  - General description of function
  - **other scripts called, usually from the runscripts directory**
  - *cluster request step name* (From the configs/cluster_request.csv file, sets the cluster job parameters. Useful for out of memory or time errors)
  - Other details about this step, e.g., functions it wraps, output or input files
  - <u>outfiles</u> are checked before the step is run. If present, that steps.py function is NOT run (unless the –overwrite flag is present)

For a graphical depiction of the full workflow (from Laura DiNicola) see the last two pages of this document.

---

<u>SETUP</u>

- **steps.xnat_to_nii_gz_fieldmap**
  - Pulls data using xnat. Prepares fieldmap dicoms for use in fm_unw.sh. Can handle both double echo and reverse plane encode fieldmaps
  - *cluster request step: ingest_fieldmap*
  - <u>For gradient double echo</u> (FUGUE, mag and phase)
    - **runscript/xnat_to_nii_gz_fm.sh**
    - ArcGet.py for mag and phase images
    - dcm2niix: mag_imag.nii.gz, pha_img.nii.gz
    - **runscript/fmap_fsl_prepare_fieldmap_prep.sh**
      - bet: mag_img_brain
      - fslmaths to erode the mag brain image: mag_img_brain_ero
      - fsl_prepare_fieldmap: {sessionid}_{#}_fieldmap.nii.gz, {sessionid}_{#}_mag_img_brain_mask.nii.gz
  - <u>For opposite encoded single echo</u> (TOPUP, AP and PA)
    - **runscript/xnat_to_nii_gz_topup.sh**
    - ArcGet.py for both field maps
    - dcm2niix: mag_imag.nii.gz, pha_img.nii.gz
    - Also pulls TotalReadoutTime from sidecar to make topupDatain.dat file.
    - **runscript/fmap_topup_prep.sh**
      - check unwarp images are the same size
      - If main output dir (OUTDIR in the subject_lists/{subj}.cfg file) is named *_AP then order scans AP/PA, opposite order if named *_PA
      - topup: topup_fmap.nii.gz, se_epi_unwarped.nii.gz, topup_out
      - fslmaths to convert to radians: {sessionid}_{#}_fieldmap.nii.gz
      - fslmaths to create mag image from se_epi_unwarped: mag_img.nii.gz
      - bet for brain mask: mag_img_brain.nii.gz
      - fslmaths erode: mag_img_brain_ero
  - Also uses dss.sh (delayed start spooler) for dcm2niix calls

    - <u>outfiles</u>: NAT/<ses>/FMAP_<#>/<ses>_<#>_fieldmap.nii.gz
- **steps.xnat_to_nii_gz_anat**
    - Pulls anatomical with xnat
    - **runscript/xnat_to_nii_gz_anat.sh**
    - *cluster request step: ingest_anat*
    - ArcGet to scratch
    - dss.sh (delayed start spooler) in same dir as iProc.py
    - dcm2niix (into BIDS, .nii.gz) -> tmp1.nii.gz
    - give error if it's not in LPI
    - fslmaths to float
    - rsync to NAT dir: {ses}_mpr{#}_reorient.nii.gz
    - <u>outfiles</u>:
        - NAT/<ses>/ANAT_<#>/<ses>_mpr<#>.nii.gz
        - NAT/<ses>/ANAT_<#>/<ses>_mpr<#>_reorient.nii.gz
- **steps.xnat_to_nii_gz_task**
    - Pulls functional data via xnat
    - **runscript/xnat_to_nii_gz_task.sh**
    - *cluster request step: ingest_task*
    - pull NAT/<ses>_bld_<#>_reorient_skip, echoTime (echo time in seconds) and dwellTime (effective echo spacing)
    - ArcGet to a scratch dir
    - check that number of volumes in ArcGet-ed file = NUMVOL + SKIP (from configs/tasktype_consolidated.csv)
    - get echo time from json file
    - Convert to float
    - trim extra timepoints (aka the skip)
    - <u>outfiles</u>: NAT/<ses>/<taskname>_<#>/<ses>_bld<#>_reorient_skip.nii.gz
- **steps.anat_from_bids**
    - Prepares data for iProc from a BIDS directory
    - *cluster request step: ingest_anat*
    - **runscript/ anat_from_bids.py**
    - fslmaths to force to float
    - fsloreint to force to radiological: _reorient.nii.gz
    - <u>outfiles</u>: NAT/<ses>/ANAT_<#>/<ses>_mpr<#>_reorient.nii.gz
- **steps.func_from_bids**
    - Prepares data for iProc from a BIDS directory
    - *cluster request step: ingest_task*
    - **runscript/ func_from_bids.py**
    - fslmaths to force to float
    - fsloreint to force to radiological: _reorient.nii.gz
    - fslroi to remove the first N volumes (as specified in configs/tasktype_consolidated.csv): {ses}_bld{run_zpad}_reorient_skip.nii.gz'
    - *NOTE: Unlike the xnat version, this does NOT check the that the number of volumes = NUMVOL + SKIP from configs/tasktype, so this might cause an error in combine_and_apply_warps*
    - <u>outfiles</u>:

        - NAT/<ses>/<taskname>_<#>/<ses>_bld<#>_reorient_skip.nii.gz
        - NAT/<ses>/<taskname>_<#>/<ses>_bld<#>_echoTime.sec
        - NAT/<ses>/<taskname>_<#>/<ses>_bld<#>_dwellTime.sec
- **steps.fmap_from_bids**
    - Prepares data for iProc from a BIDS directory. Prepares fieldmap for use in fm_unw.sh. Can handle both double echo and reverse plane encode fieldmaps
    - *cluster request step: ingest_fieldmap*
    - <u>For gradient double echo</u> (FUGUE, mag and phase)
        - **runscript/ fmap_from_bids.py**
        - fslmerge if multiple mag or phase images
        - bet: mag_img_brain
        - fslmaths to erode the mag brain image: mag_img_brain_ero
        - fsl_prepare_fieldmap: {sessionid}_{#}_fieldmap.nii.gz, {sessionid}_{#}_mag_img_brain_mask.nii.gz
    - <u>For opposite encoded single echo</u> (TOPUP, AP and PA)
        - Pulls TotalReadoutTime from sidecar to make topupDatain.dat file.
        - **runscript/ fmap_from_bids_topup.sh**
            - Also pulls encoding direction from sidecar to make topupDatain.dat file.
        - **runscript/fmap_topup_prep.sh**
        - check unwarp images are the same size
        - If main output dir (OUTDIR in the subject_lists/{subj}.cfg file) is named *_AP then order scans AP/PA, opposite order if named *_PA, same for LR or RL
        - topup: topup_fmap.nii.gz, se_epi_unwarped.nii.gz, topup_out
        - fslmaths to convert to radians: {sessionid}_{#}_fieldmap.nii.gz
        - fslmaths to create mag image from se_epi_unwarped: mag_img.nii.gz
        - bet for brain mask: mag_img_brain.nii.gz
        - fslmaths erode: mag_img_brain_ero
    - Also uses dss.sh (delayed start spooler) for dcm2niix calls
    - <u>**outfiles**</u>: NAT/<ses>/FMAP_<#>/<ses>_<#>_fieldmap.nii.gz
- **steps.recon_all**
    - Runs Freesurfer's recon-all
    - **runscript/recon_all.sh**
    - *cluster request step: recon_all*
    - input: NAT/{sessionid}_mpr{anatno}_reorient.nii.gz, converts to orig/001.mgz
    - writes to scratch, then moves to subjects dir
    - <u>**outfiles**</u>:
        - <fsdir>/<sub>/mri/T1.mgz
        - <fsdir>/<sub>/surf/lh.pial
- **QC_fmap** (in iProc.py NOT steps)
    - *cluster request step: fmap_qc*
    - PDF of each fieldmap with an associated functional
    - <u>**Not outfiles, but produces:**</u> QC/<sub>_fmap_sag.pdf and _ax.pdf

<u>BET</u>

- **check_bet (iProc.py) -> steps.sesst_prep**

    - **runscript/sesst_prep.sh**
    - *cluster request step: sesst_prep*
    - link template dir/<ses>_mpr.nii to mpr_reorient (same for _mpr_reorient_brain and mpr_reorient_brain_mask)
    - copy SUBJECTS_DIR/mri/T1.mgz (1x1x1) to template dir/<ses>_mpr.nii.gz and make sure in MNI orientation
    - create template dir/${SESST}_mpr_reorient_brain_mask.nii.gz (from SUBJECTS_DIR/mri/brainmask.mgz: convert, fslreorient, binarize)
    - copy and fslreorient2std <ses>_mpr_brain to <ses>_mpr_reorient_brain
- **mpr_bet (iProc.py)**
    - runs bet on the <ses>_reorient_brain we just made
- *Note: these are all in 1x1x1 mm3 space (because this is how it is from Freesurfer recon), stored in cross_session_maps/templates*
- <u>Not outfiles, but produces:</u>
    - cross_section_maps/template/<ses>_mpr.nii.gz
    - cross_section_maps/template/<ses>_mpr_brain.nii.gz
    - cross_section_maps/template/<ses>_mpr_reorient.nii.gz
    - cross_section_maps/template/<ses>_mpr_reorient_brain.nii.gz
    - cross_section_maps/template/<ses>_mpr_reorient_brain_mask.nii.gz
    - cross_section_maps/mpr.nii.gz [link]
    - cross_section_maps/mpr_reorient.nii.gz [link]
    - cross_section_maps/mpr_reorient_brain.nii.gz [link]
    - cross_section_maps/mpr_reorient_brain_mask.nii.gz [link]

<u>UNWARP_AND_MOTION_CORRECT</u>

- check that cross_session_maps/templates/mpr_reorient_brain.nii.gz exists
- **steps.fsl_reorient_skip**
    - Extract template volume – as specified in cfg, MIDVOL_SESS, BOLDNO, VOLNO – with fslroi
    - **runscript/fslroi_reorient_skip[_ME].sh**
    - *cluster request step: fsl_reorient_skip*
    - <u>outfiles</u>: session_maps/templates/<sub>_D01_<midvol task>_bld<midvol run #>_midvol.nii.gz
    - Also pulls echoTime_e1.sec, dwellTime_e1.sec, and _e1.json from NAT dir to cross_section_maps/templates
- **steps.fm_unwarp_midvol**
    - Extract middle volume of scan from Session1 acquired closest in time to the field map and unwarp midvol image using fieldmap
    - **runscript/fm_unw.sh**
    - *cluster request step: fm_unwarp_midvol*
    - <u>outfiles</u>: cross_session_maps/templates/{sub}D01_{bold_name}_midvol_unwarp.nii.gz
    - Many things with fslmaths and fugue and warping in the cross_session_maps/templates/fm_unwarp<midvol run #> directory
    - If no fieldmap, this just copies

    - NOTE: weird things happened with passing parameteres to .sh file. Mad that there's no rmfiles? Removed this parameter from bash file since there are no rmfiles for this step.
- **steps.create_upsampled_midvoltarget**
    - Upsample to create midvol target
    - **runscript/create_upsampled_midvol_target.sh**
    - *cluster request step: create_upsampled_midvol_target*
    - [used to use fslcreatehd: create header with specific voxel dimensions but this wasn't flexible]
    - Flirt with -applyisoxfm of 1.2 or 2. Ref is midvol_unwarp: cross_session_maps/templates/{sub}_D01_${bold_name}_midvol_unwarp.nii.gz
    - <u>outfiles</u>: cross_session_maps/templates/{sub}_D01_{midvol_bold_name}_midvol_unwarp_1p2i.nii.gz or _2mm.nii.gz
    - copy to cross_session_maps/templates /${sub}_midvol_unwarp_1p2i.nii.gz or _2mm.nii.gz

- **steps.fm_unwarp_and_mc_to_midvol**
    - Decide on each run's "middle volume"
        - Using fsl_motion_outliers: note any volumes whose framewise displacement is greater than the threshold in the subject configuration file (in mm); note those in outlier_num.txt
        - If the middle volume of the run is a motion outlier, check the next 5 volumes. Use the first one that is not an outlier as the new "middle" volume. If they are all outliers, throw an error.
    - Grab middle volume, motion correct each run with mcflirt, and register all runs to unwarped midvol target produced from the midvol of midvol_sess specified in conf
    - Do this for each echo based on the MC from echo 1
    - *cluster request step: fm_unwarp_and_mc_to_midvol*
    - **runscript/fm_unwarp_and_mc_to_midvol.sh ->**
        - **runscript/create_motion_outlier_matrix.py, p2a.sh**
        - pull midvol (as specified in config file)
        - Use fsl_motion_outliers to save to text file any motion outliers beyong the threshold specified in the config file; warn if run has > 20% of outlier volumes. If specified midvol is a motion outlier, look for a non-outlier "mid" vol 5 volumes within the specified one. If one exists, use that one, otherwise gives an error (Thanks STAR group!)
        - Create the motion outlier matrix
        - Mcflirt the midvol run
        - If a field map is available, unwarp the midvol, otherwise, the midvol will be called unwarped but it isn't!
        - Flirt the BOLD run to the unwarped (or "unwarped") mid vol
        - p2a.sh reformats the mcflirt par motion correct regressors
    - **runscript/fm_unw.sh**
        - prepares fieldmap (but not using fsl_prepare_fieldmap, does it manually)
        - FUGUE
        - convertwarp and applywarp

- **runscript/p2a.sh:**
    - Looks at regressors file,
    - Skip comment lines (lines starting with #).
    - Check all rows have the same number of columns.
    - Reorder the columns using a modulo operation.
    - OUTPUT: new file (${OUTFILE), where each row is formatted with a sequential index, the data columns, and an additional 1 at the end of each row.
- <u>outfiles</u>:
    - NAT/<ses>/<taskname>_<#>/<ses>_bld<#>_reorient_skip_mc_unwarp _midvol_to_midvoltarg.nii.gz'
    - NAT/<ses>/<taskname>_<#>/ <#>_to_midvoltarg.mat')
    - NAT/<ses>/<taskname>_<#>/ <ses>_bld<#>_reorient_skip_mc.mat/MAT_0...
    - If fieldmaps, NAT/<ses>/<taskname>_<#>/fm_unwarp><#>/EF_UD_warp.nii.gz

- grab files with merging a glob with reorient_skip_mc_unwarp_midvol_to_midvoltarg
- **steps.fslmerge_meantime**
    - fslmerge in time dimension, then fslmaths -Tmean to get the mean across time at each voxel
    - aka, create mean BOLD midvol template produced by averaging the output of fm_unwarp_and_mc_to_midvol
    - **runscript/fslmerge_meantime.sh**
    - *cluster request step: fslmerge_meantime_midvols*
    - <u>outfiles</u>: cross_session_maps/templates/<sub>_midvol_unwarp_<res>mm.nii.gz.nii.gz
- **steps.align_to_midvol_mean**
    - flirt to calculate warp from each reorient_skip_mc_midvol_unwarp to mean template
    - **runscript/align_to_midvol_mean.sh**
    - *cluster request step: align_to_midvol_mean*
    - <u>outfiles</u>:
        - NAT/<ses>/<taskname>_<#>/<sessionid>_<bold_no>_on_midmean.nii.gz
        - NAT/<ses>/<taskname>_<#>/<sessionid>_<bold_no>_to_allscans.mat
- **QC_unwarp_motioncorrect_align**
    - check mean of bold midvol and all the fm_unwarp_and_mc_to_midvol outputs
    - *cluster request step: midvol_qc*
    - <u>Not outfiles, but produces:</u> QC/<sub>_reorient_skip_mc_unwarp_midvol_to_midvoltarg_ax.pdf and _sag.pdf

<u>T1_WARP_AND_MASK</u>

- **steps.bbreg**
    - Freesurfer's bbregister (within subect linear reg)
    - **runscript/bbreg.sh**
    - *cluster requests step: bbreg*
    - <u>outfiles</u>:
        - cross_session_maps/templates/ <sub>_allscans_meanBOLD_to_T1.dat
        - cross_session_maps/templates/ <sub>_allscans_meanBOLD_to_T1.mat

- **steps.compute_T1_MNI_warp**
    - Outputs to cross_session_maps/templates/
    - **runscript/compute_T1_MNI_warp.sh**
    - *cluster requests step: computer_ T1_ MNI_ warp*
    - fslswpdim -> mpr_brain.nii.gz
    - flirt mpr_brain to mpr_brain_mni (vol and .mat)
    - fnirt mpr -> mpr_brain_to_mni.mat, anat_mni_underlay, mpr_to_mniFNIRT.mat
    - invwarp -> TEMPLATEDIR/MNI_to_<ses>_mni_underlay.mat.nii.gz
    - applywarp -> anat_mni_underlay_brain
    - bin above for anat_mni_underlay_brain_mask
    - <u>outfiles</u>: cross_session/templates/MNI_to_<T1 #>_mni_underlay.mat.nii.gz
- **steps.reg_MNI_CSF_WM_to_T1**
    - **runscript/reg_MNI_CSF_WM_to_T1.sh**
    - *cluster requests step: reg_ MNI_ CSF_ WM_ to_ T1*
    - make relative links to
        - cross_section_maps/templates/mni_masks/avg152T1_brain_MNI and WM and ventricles
        - cross_section_maps/templates/mni_masks/wm_mask.nii.gz and csf_...
    - upsample masks to 1mm space (same dir just with _1mm affix)
    - applywarp to project those 2 masks to T1 space: wm_mask_mpr.nii.gz and csf...
    - link session brain mask to mni_masks/wb_mask_mpr_reorient.nii.gz
    - fslreorient2std -> mni_masks/wm_mask_mpr_reorient.nii.gz and csf...
    - <u>outfiles</u>:
        - cross_session/templates/mni_masks/csf_mask_mpr_reorient.nii.gz
        - cross_session/templates/mni_masks/wm_mask_mpr_reorient.nii.gz

<u>COMBINE_AND_APPLY_WARP</u>

- Combines warps to move data once to MNI and T1 1 or 2mm^3 spaces – MNI### and NAT### directories - and QC
- check that cross_session_maps/templates/anat_mni_underlay_brain.nii.gz exists
- **steps.size_brainmask**
    - **runscript/size_brainmask.sh**
    - *cluster request step: size_ brainmask*
    - dilate mpr_reorient_brain_mask by 10 voxels -> mpr_reoeient_brain_mask_dil10
    - same for anat_mni_underlay_brain_mask -> anat_mni_underlay_brain_mask_dil10.nii.gz
    - <u>outfiles</u>:
        - cross_session/templates/anat_mni_underlay_brain_mask_dil10.nii.gz
        - cross_session/templates/mpr_reorient_brain_mask_dil10.nii.gz
- **steps.combine_warps_parallel** (combine warps, project to T1 or MNI space)
    - **runscript/combine_warps_parallel[_ME].sbatch** (added echo num flag to run across all echos, changed save dir to include _e?)

        - **iProc_p4_sbatch_combined[_ME].py** (in main iProc dir, NOT runscript dir)(Added echo number flag and changed the convertwarp output to .nii.gz instead of .mat) to run ...
    - *cluster request step: combine_warps_parallel_anat* or *combine_warps_parallel_mni*
    - create <boldno>_to_anat.mat (from <boldno>_to_allscans.mat + <boldno>_meanBOLD_to_<T1 or MNI>.mat)
    - convert_warpcall_anat: convertwarp creates <boldno>_<T1 or MNI>/<T1 or MNI>_TARG_WARP_<volno>.nii.gz
    - apply_warpcall_anat: applywarp creates <boldno>_<T1 or MNI>_TARG_FILE_<volno>.nii.gz
    - <u>outfiles</u>:
        - NAT/<ses>/<taskname>_<bold #>/<bold #>_<T1 or MNI>/<T1 or MNI>_TARG_FILE_<vol num>
    -
- **steps.combine_warps_post**
    - **runscript/combine_warps_post.sh**
    - *cluster request step: combine_warps_post_anat*
    - fslmerge all T1_TARG_WARP files: <ses>_bold<boldno>_reorient_skip_mc_unwarp_anat_tmp.nii.gz
    - fslreorient2std above output: <ses>_bold<boldno>_reorient_skip_mc_unwarp_anat.nii.gz
    - multiply above by mrpr_reorient_brain_mask_dil10.nii.gz (in template dir)
    - fslmaths Tmean: <ses>_bold<boldno>_reorient_skip_mc_unwarp_anat_mean.nii.gz
    - fslroi for just the midvol: <ses>_bold<boldno>_reorient_skip_mc_unwarp_anat_vol<midvol>.nii.gz
    - <u>outfiles</u>:
        - NAT<res>/<ses>/<taskname>_<bold>/<ses>_bld<#>_reorient_skip_mc_unwarp_anat_vol<midvol #>.nii.gz
        - NAT<res>/<ses>/<taskname>_<bold>/<ses>_bld<#>_reorient_skip_mc_unwarp_anat.nii.gz
        - NAT<res>/<ses>/<taskname>_<bold>/<ses>_bld<#>_reorient_skip_mc_unwarp_anat_mean.nii.gz
- **steps.combine_warps_post_MNI**
    - **runscript/ combine_warps_post_MNI.sh**
    - *cluster request step: combine_warps_post_mni*
    - fslmerge all MNI_TARG_WARP files: <ses>_bold<boldno>_reorient_skip_mc_unwarp_mni_tmp.nii.gz
    - fslreorient2std above output: <ses>_bold<boldno>_reorient_skip_mc_unwarp_mni.nii.gz
    - multiply above by mrpr_reorient_brain_mask_dil10.nii.gz (in template dir)
    - fslmaths Tmean: <ses>_bold<boldno>_reorient_skip_mc_unwarp_mni_mean.nii.gz
    - fslroi for just the midvol: <ses>_bold<boldno>_reorient_skip_mc_unwarp_mni_vol<midvol>.nii.gz
    - <u>outfiles</u>:
        - MNI<res>/<ses>/<taskname>_<bold>/<ses>_bld<#>_reorient_skip_mc_unwarp_anat_mni_vol<midvol #>.nii.gz

        - MNI<res>/<ses>/<taskname>_<bold>/<ses>_bld<#>_reorient_skip_mc_unwarp_anat_mni.nii.gz
        - MNI<res>/<ses>/<taskname>_<bold>/<ses>_bld<#>_reorient_skip_mc_unwarp_anat_mni_mean.nii.gz
        -
- QC
    - merge glob to look for
        - NAT###/.../...reorient_skip_mc_unwarp_anat_mean
        - NAT###/.../...reorient_skip_mc_unwarp_anat_vol
        - MNI###/.../...reorient_skip_mc_unwarp_anat_mni_mean
    - steps.fslmerge_meantime (anat mean, then anat_midvols, mni mean, mni_midvols)
        - runscript/fslmerge_meantime.sh
    - *cluster request steps: fslmerge_meantime_anat_mean, anat_mean_qc, fslmerge_meantime_anat_midvols, anat_midvols_qc, fslmerge_meantime_mni_mean*
    - <u>Not outfiles, but produces</u>: pdf and sh files <sub>_file names from above_ax or sag

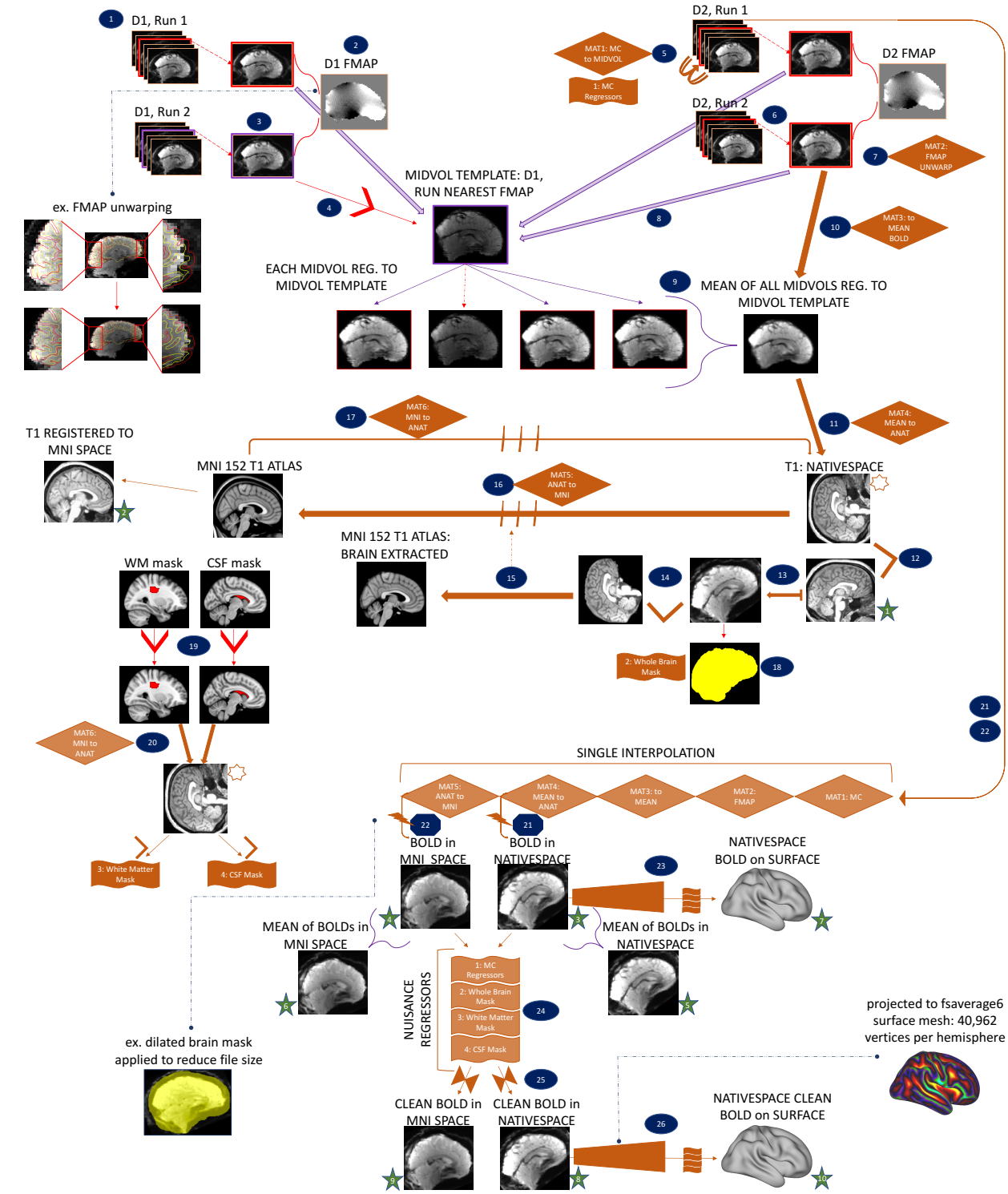
- *WARNING: sometimes this step fails for ME scans because it can't access a file that's definitely there. Maybe different echo jobs trying access the file at the same time? If this happens, just run again.*
- *Also: the T1_TARG_WARP directory can get really big. Suggest rm after done with this step even if not removing files.*

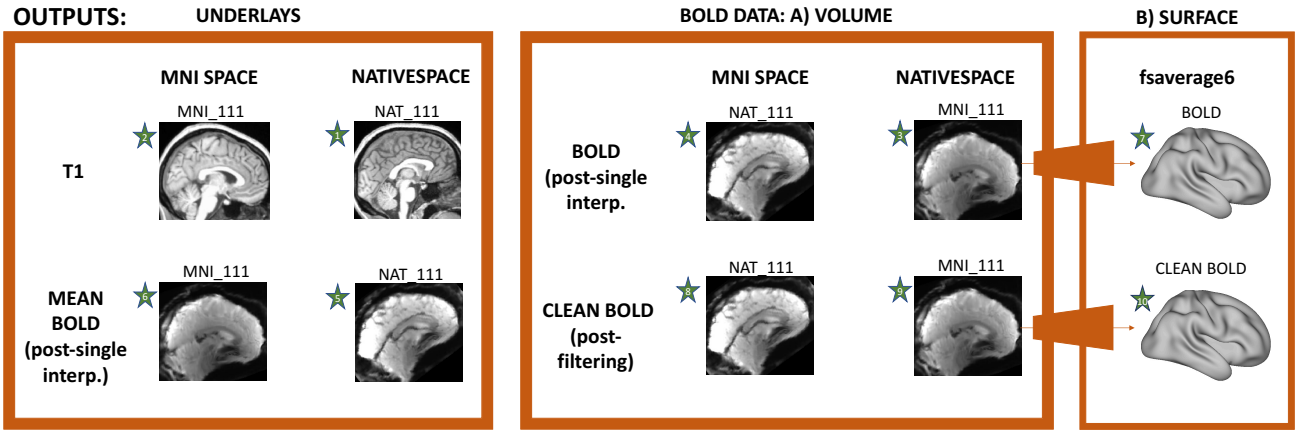
**[For multi-echo sequences, run TEDANA here.]**

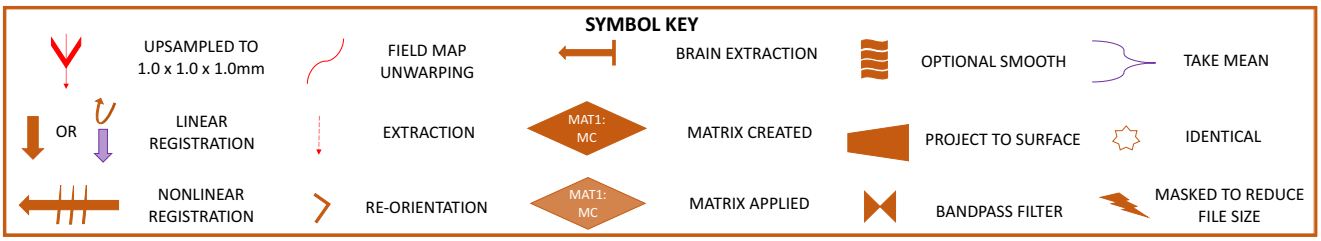
<u>FILTER_AND_PROJECT</u>

- **steps.calculate_nuisance_param** (x2 for MNI### and NAT###)
    - **runscript/calculate_nuisance_params.sh**
    - *cluster request step: calculate_nuisance_params*
    - fslmeants (outputs mean itensity across voxels overtime, text file) with CSF, WM and WB (whole brain) masks
    - combine with motion correction parameters
    - 3 sets: detrended and zscore-d, difference scores from that, squares of the difference scores
    - <u>outfiles</u>: <ses>_bld{<#>_reorient_skip_mc_unwarp_anat_nuis.dat [contains columns for 6 motion correction nuisance parameters, with temporal derivatives and quadratric terms, followed by the CSF, white matter and whole-brain parameters and outlier regressors]
    - <u>also outputs</u>: nuis_36P is same as above but without the outlier regressor, PHYS combines the WM, GM, and CSF only, NUIS_TS combines motion correction and PHYS
- **steps.nuisance_regress (MNI### and NAT###)**
    - **runscript/nuisance_regress.sbatch**
    - *cluster request steps: nuisance_regress_mni, nuisance_regress_anat*
    - 3dTproject (afni) detrends out anat_nuis.dat from calculate_nuisance_param step
    - <u>outfiles</u>:
        - <ses>_bld<#>_reorient_skip_mc_unwarp_anat_resid.nii.gz

        - <ses>_bld<#>_reorient_skip_mc_unwarp_anat__mni_resid.nii.gz
        - <ses>_bld<#>_reorient_skip_mc_unwarp_anat_resid+orig.BRIK
        - <ses>_bld<#>_reorient_skip_mc_unwarp_anat_resid+orig.HEAD
          <ses>_bld<#>_reorient_skip_mc_unwarp_anat_mni_resid+tlrc.BRIK
        - <ses>_bld<#>_reorient_skip_mc_unwarp_anat_mni_resid+tlrc.HEAD
        - orig = NAT, tlrc = MNI
- **steps.bandpass** (MNI### and NAT###) *0.01-0.1Hz, masked with mpr_reorient_brain_mask.nii.gz*
    - **runscript/bandpass[_ME].sbatch**
    - *cluster request steps: bandpass_mni, bandpass_anat*
    - 3dcopy (afni) to put into afni format
    - 3dBandpass (afni)
    - 3dcopy: MNI### or NAT###/{sessionid}_bld{bold_no}_desc-denoised_bold_afni+(orig or tldr).BRIK/.HEAD
    - **outfiles**: <ses>_bld<#>_reorient_skip_mc_unwarp_anat_resid_bpss.nii.gz
- **steps.fs6_project_to_surface**
    - **runscript/fs6_project_to_surface[_ME].sh**
    - *cluster request step: fs6_project_to_surface*
    - mri_vol2surf to project bandpass filtered (..._reorient_skip_mc_unwarp_anat_resid_bpss_fsaverage6.nii.gz) to surface
        - OUTPUT: FS6/.../lh.{ses}_{bold_no}_reorient_skip_mc_unwarp_anat__fsaverage6.nii.gz and rh...
    - mri_surf2surf to smooth output from mri_vol2surf above
        - OUTPUT: _sm<smoothing kernal>.nii.gz
    - **outfiles**: <FS6DIR>/<ses>/<task>/lh.resid_bpss_fsaverage6_sm<kernel>.nii.gz
        - also projects: wb_resid and just resid


D1, Run 1
D1 FMAP
D1, Run 2
ex. FMAP unwarping
MIDVOL TEMPLATE: D1, RUN NEAREST FMAP
EACH MIDVOL REG. TO MIDVOL TEMPLATE
MEAN OF ALL MIDVOLS REG. TO MIDVOL TEMPLATE
D2, Run 1
D2 FMAP
D2, Run 2
MAT1: MC to MIDVOL
1: MC Regressors
MAT2: FMAP UNWARP
MAT3: to MEAN BOLD
MAT4: MEAN to ANAT
MAT5: ANAT to MNI
MAT6: MNI to ANAT
T1 REGISTERED TO MNI SPACE
MNI 152 T1 ATLAS
T1: NATIVESPACE
MNI 152 T1 ATLAS: BRAIN EXTRACTED
WM mask
CSF mask
2: Whole Brain Mask
3: White Matter Mask
4: CSF Mask
SINGLE INTERPOLATION
MAT5: ANAT to MNI
MAT4: MEAN to ANAT
MAT3: to MEAN
MAT2: FMAP
MAT1: MC
BOLD in MNI SPACE
BOLD in NATIVESPACE
NATIVESPACE BOLD on SURFACE
MEAN of BOLDs in MNI SPACE
MEAN of BOLDs in NATIVESPACE
NUISANCE REGRESSORS
1: MC Regressors
2: Whole Brain Mask
3: White Matter Mask
4: CSF Mask
ex. dilated brain mask applied to reduce file size
projected to fsaverage6 surface mesh: 40,962 vertices per hemisphere
CLEAN BOLD in MNI SPACE
CLEAN BOLD in NATIVESPACE
NATIVESPACE CLEAN BOLD on SURFACE


OUTPUTS:
UNDERLAYS
MNI SPACE
NATIVESPACE
T1
MNI_111
NAT_111
MEAN BOLD (post-single interp.)
MNI_111
NAT_111
BOLD DATA: A) VOLUME
B) SURFACE
MNI SPACE
NATIVESPACE
fsaverage6
BOLD (post-single interp.)
NAT_111
MNI_111
BOLD
CLEAN BOLD (post-filtering)
NAT_111
MNI_111
CLEAN BOLD


SYMBOL KEY
UPSAMPLED TO 1.0 x 1.0 x 1.0mm
FIELD MAP UNWARPING
BRAIN EXTRACTION
OPTIONAL SMOOTH
TAKE MEAN
OR
LINEAR REGISTRATION
EXTRACTION
MAT1: MC
MATRIX CREATED
PROJECT TO SURFACE
IDENTICAL
NONLINEAR REGISTRATION
RE-ORIENTATION
MAT1: MC
MATRIX APPLIED
BANDPASS FILTER
MASKED TO REDUCE FILE SIZE

1. Download dicoms for all runs across all days (D) and merge into run-specific nifti files.
2. Download and prepare field maps (FMAPs) for each day.
3. Extract the middle volume of the run acquired closest in time to the FMAP on D1. Use the D1 FMAP to unwarp this midvol.
4. Upsample the FM unwarped midvol from 2.4 x 2.4 x 2.4mm to 1.0 x 1.0 x 1.0mm resolution. This is the MIDVOL TEMPLATE.
5. For each run, calculate motion correction (MC) matrices for each volume to the run's middle volume [MAT 1]. Calculate and save the MC regressors for later use.
6. For each run, extract the middle volume.
7. For each run, use the relevant day's FMAP to unwarp the extracted middle volume. Save the matrix for each run's FMAP unwarping [MAT 2].
8. Register each run's FM-unwarped middle volume to the midvol template.
9. Merge all of the midvol-template-registered volumes (there should be one per run) and take the mean. This is now the the MEANBOLD.
10. Calculate the matrix for registering each run's FM-unwarped midvol to the meanbold [MAT 3].
11. Calculate the matrix for registering the meanbold to the NATIVESPACE T1* [MAT 4].
    * Prior to running the pipeline, Freesurfer's recon-all command is run on all T1 anatomical scans available for each subject. Each subject's T1 with the best surface registration is identified. This T1.mgz file (output by Freesurfer) is then converted to a nifti file for use within the pipeline.
    * The T1 nativespace data are in 1.0mm isotropic resolution when output by Freesurfer.
12. Reorient the nativespace template to fit the FSL standard orientation.
12. Brain extract the nativespace T1.
13. Reorient the brain-extracted nativespace T1 back to the original orientation (as output by Freesurfer).
14. Calculate the registration of the brain-extracted nativespace T1 to a brain-extracted MNI 152 Atlas T1. The resultant matrix is then input to (16).
15. Calculate the nonlinear registration of the nativespace template to the MNI TEMPLATE* [MAT 5].
    * The MNI 152 Atlas used is in 1.0mm isotropic resolution.
16. Calculate the inverse nonlinear registration of the MNI template to the nativespace template [MAT 6].
17. Create a whole brain mask specific to the subject's reoriented, brain-extracted nativespace template.
18. Upsample the standard white matter (WM) and CSF masks to 1.0 x 1.0 x 1.0mm space.
19. Applying [MAT 6], register the WM and CSF masks to the nativespace T1.*
20. For each run, split the data into individual volumes. For each volume, combine relevant matrices [MATS 1-4] and apply in a single interpolation step. Merge the output volumes, for each run; mask the output data to reduce file size; reorient to FSL standard orientation.
21. For each run, split the data into individual volumes. For each volume, combine relevant matrices [MATS 1-5] and apply in a single interpolation step. Merge the output volumes, for each run; mask the output data to reduce file size; reorient to FSL standard orientation.
22. Project the nativespace output in the volume (output from (21)) to the surface using the fsaverage6 mesh (through Freesurfer), which includes ~40,962 vertices per hemisphere. Outputs include both smoothed (2mm) and unsmoothed data.
23. For both the MNI and nativespace single interpolation outputs, regress the whole-brain, WM, CSF and motion nuisance parameters.
24. Bandpass filter the data, post-regression.
25. Project the nativespace residual, bandpass-filtered data to the surface using the fsaverage6 mesh (through Freesurfer), which includes ~40,962 vertices per hemisphere. Outputs include both smoothed (2mm) and unsmoothed data.